BỘ CÔNG THƯƠNG

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

—o0o—

BÁO CÁO ĐỒ ÁN MÔN HỌC

HỌC SÂU (DEEP LEARNING)

Đồ án số: #20 – Phát hiện spam email

Sinh viên thực hiện:	23684961 - Tống Nguyễn Nhật Tiến
	23689101 - Nguyễn Tiến Phát
Lớp:	DHKHMT19BTT
Giảng viên hướng dẫn:	TS. Lê Vũ Hạo

TP. Hồ Chí Minh, Ngày 30 tháng 3 năm 2026

Mục lục

1 Giới thiệu **5**

1.1 Bối cảnh và động cơ 5

1.2 Mục tiêu đồ án 5

1.3 Phạm vi và cấu trúc báo cáo 6

2 Cơ sở lý thuyết và nghiên cứu liên quan **7**

2.1 Kiến thức nền tảng 7

2.1.1 Word Embedding 7

2.1.2 Recurrent Neural Networks (RNN) 7

2.1.3 Long Short-Term Memory (LSTM) 8

2.1.4 Gated Recurrent Unit (GRU) 8

2.1.5 Bidirectional RNN 8

2.1.6 Attention Mechanism 8

2.1.7 Các kỹ thuật Regularization 9

2.2 Các nghiên cứu hoặc mô hình liên quan 9

3 Bài toán và dữ liệu **10**

3.1 Mô tả bài toán 10

3.2 Dataset 10

3.2.1 Nguồn dữ liệu 10

3.2.2 Tiền xử lý dữ liệu 12

4 Phương pháp đề xuất **14**

4.1 Mô hình 1 – Baseline: BiLSTM + Embedding + Pooling 14

4.2 Mô hình 2 – Nâng cao: BiGRU + Attention 16

4.3 Hàm mất mát và thuật toán tối ưu 17

4.3.1 Hàm mất mát 17

4.3.2 Thuật toán tối ưu 17

4.3.3 Learning Rate Scheduler 17

4.3.4 Callbacks 18

5 Thực nghiệm và đánh giá **19**
5.1 Thiết lập thí nghiệm 19
5.2 Kết quả định lượng 20
5.2.1 Learning Curves 20
5.2.2 Confusion Matrix và PR Curve 21
5.2.3 Bảng so sánh tổng hợp 22
5.2.4 Biểu đồ so sánh tổng hợp 22
5.2.5 Số lượng tham số 23
5.3 Phân tích kết quả (CLO3) 23
5.3.1 So sánh Optimizer: Adam vs SGD 23
5.3.2 So sánh Regularization: Dropout vs Weight Decay 23
5.3.3 So sánh Baseline vs Nâng cao 23
5.3.4 Phân tích lỗi điển hình 24
5.3.5 Giới hạn của mô hình 24

6 Kết luận và hướng phát triển **26**
6.1 Kết luận 26
6.2 Hướng phát triển 26

Danh sách hình vẽ

3.1 Phân bố số lượng mẫu theo lớp Ham/Spam trong dataset 11
3.2 Pipeline tiền xử lý dữ liệu email 12

4.1 Kiến trúc mô hình 1 – BiLSTM Baseline 14
4.2 Kiến trúc mô hình 2 – BiGRU + Attention (Nâng cao) 16

5.1 Learning curves của mô hình BiLSTM Baseline (Adam + Dropout) . . 20
5.2 Learning curves của mô hình BiGRU + Attention (Adam + Dropout) . 21
5.3 Confusion Matrix và Precision-Recall Curve – BiLSTM Baseline 21
5.4 Confusion Matrix và Precision-Recall Curve – BiGRU + Attention . . . 22
5.5 Biểu đồ so sánh tổng hợp tất cả các mô hình và thí nghiệm 22
5.6 Phân bố xác suất dự đoán: phân loại đúng vs sai, FP vs FN 24

Danh sách bảng

3.1 Thông tin tổng quan dataset 11
3.2 Phân chia tập dữ liệu (stratified split) 13

5.1 Thiết lập thí nghiệm 19
5.2 So sánh kết quả tất cả các mô hình và thí nghiệm 22
5.3 So sánh số lượng tham số của hai mô hình 23

Chương 1

Giới thiệu

1.1 Bối cảnh và động cơ

Trong kỷ nguyên số hóa, email vẫn là một trong những phương tiện giao tiếp quan trọng nhất trong cả môi trường doanh nghiệp lẫn cá nhân. Theo thống kê, hàng ngày có khoảng 300 tỷ email được gửi đi trên toàn cầu, trong đó gần 45–50% là thư rác (spam). Spam email không chỉ gây phiền toái mà còn tiềm ẩn nhiều rủi ro bảo mật nghiêm trọng: lừa đảo tài chính (phishing), phát tán mã độc (malware), đánh cắp thông tin cá nhân và chiếm đoạt tài khoản.

Các phương pháp lọc spam truyền thống dựa trên quy tắc (rule-based) hoặc học máy cổ điển (Naive Bayes, SVM) đã bộc lộ nhiều hạn chế khi đối mặt với các kỹ thuật spam ngày càng tinh vi. Spam hiện đại sử dụng nhiều chiến thuật né tránh: biến đổi từ ngữ, chèn mã HTML phức tạp, sử dụng hình ảnh thay vì văn bản, và mạo danh các tổ chức uy tín.

Deep Learning, với khả năng tự động học các đặc trưng phức tạp từ dữ liệu văn bản, mang đến một hướng tiếp cận mạnh mẽ hơn cho bài toán phân loại spam. Các mô hình sequence như LSTM, GRU kết hợp cơ chế Attention có thể nắm bắt được ngữ cảnh và mối quan hệ ngữ nghĩa giữa các từ trong email, từ đó phát hiện spam hiệu quả hơn.

1.2 Mục tiêu đồ án

Đồ án hướng đến việc đạt được các chuẩn đầu ra (CLO) sau:

- **CLO1 – Phân tích và thiết kế:** Phân tích bài toán phân loại nhị phân spam/ham, nhận diện các thách thức đặc thù của dữ liệu email (HTML tags, ký tự đặc biệt, mất cân bằng lớp), và lựa chọn kiến trúc Deep Learning phù hợp.

- **CLO2 – Triển khai và thực nghiệm:** Triển khai đầy đủ pipeline huấn luyện bao gồm tiền xử lý dữ liệu, xây dựng và huấn luyện ít nhất hai mô hình (BiLSTM baseline và BiGRU + Attention nâng cao), so sánh các cấu hình optimizer và regularization.
- **CLO3 – Đánh giá và vận dụng:** Đánh giá kết quả bằng các metric phù hợp (Precision, Recall, F1-Score), phân tích lỗi điển hình, thảo luận giới hạn mô hình và đề xuất hướng cải tiến.

1.3 Phạm vi và cấu trúc báo cáo

Phạm vi đồ án:

- Sử dụng dataset Enron Spam (spam_ham_dataset.csv) với 5170 mẫu email.
- Triển khai hai mô hình: BiLSTM (baseline) và BiGRU + Attention (nâng cao).
- Thực hiện các thí nghiệm so sánh optimizer (Adam vs SGD) và regularization (Dropout vs Weight Decay).
- Xây dựng hệ thống demo phân loại email real-time.

Cấu trúc báo cáo:

- **Chương 1:** Giới thiệu bối cảnh, mục tiêu và phạm vi.
- **Chương 2:** Cơ sở lý thuyết về các kiến trúc Deep Learning.
- **Chương 3:** Mô tả bài toán, dataset và tiền xử lý dữ liệu.
- **Chương 4:** Phương pháp đề xuất – kiến trúc hai mô hình.
- **Chương 5:** Thực nghiệm, kết quả và phân tích.
- **Chương 6:** Kết luận và hướng phát triển.

Chương 2

Cơ sở lý thuyết và nghiên cứu liên quan

2.1 Kiến thức nền tảng

2.1.1 Word Embedding

Word Embedding là kỹ thuật biểu diễn từ vựng dưới dạng vector số thực trong không gian nhiều chiều. Thay vì sử dụng biểu diễn one-hot encoding (thưa và không chứa thông tin ngữ nghĩa), embedding layer học được các biểu diễn dày đặc (dense) trong quá trình huấn luyện.

Cho vocabulary V với $|V|$ từ, embedding layer ánh xạ mỗi từ w_i thành vector $\mathbf{e}_i \in \mathbb{R}^d$:

$$\mathbf{e}_i = \mathbf{E} \cdot \text{one_hot}(w_i), \quad \mathbf{E} \in \mathbb{R}^{d \times |V|} \tag{2.1}$$

trong đó d là kích thước embedding (trong đồ án này $d = 128$).

2.1.2 Recurrent Neural Networks (RNN)

RNN là kiến trúc mạng nơ-ron được thiết kế để xử lý dữ liệu tuần tự. Tại mỗi bước thời gian t, RNN tính toán hidden state:

$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h) \tag{2.2}$$

trong đó $\mathbf{x}_t$ là input tại thời điểm t, $\mathbf{h}_{t-1}$ là hidden state trước đó, và σ là hàm kích hoạt.

Tuy nhiên, RNN cơ bản gặp vấn đề *vanishing/exploding gradient* khi xử lý chuỗi dài, dẫn đến sự ra đời của LSTM và GRU.

2.1.3 Long Short-Term Memory (LSTM)

LSTM [2] giải quyết vấn đề gradient bằng cơ chế cổng (gate mechanism) với ba cổng chính:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(Forget gate)} \tag{2.3}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(Input gate)} \tag{2.4}$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \quad \text{(Candidate)} \tag{2.5}$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad \text{(Cell state)} \tag{2.6}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{(Output gate)} \tag{2.7}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad \text{(Hidden state)} \tag{2.8}$$

2.1.4 Gated Recurrent Unit (GRU)

GRU [3] là biến thể đơn giản hóa của LSTM với hai cổng (reset và update), có ít tham số hơn nhưng hiệu suất tương đương:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \text{(Update gate)} \tag{2.9}$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \text{(Reset gate)} \tag{2.10}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \text{(Candidate)} \tag{2.11}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad \text{(Hidden state)} \tag{2.12}$$

2.1.5 Bidirectional RNN

Bidirectional RNN xử lý chuỗi theo cả hai chiều (trái-sang-phải và phải-sang-trái), cho phép mỗi output phụ thuộc vào toàn bộ ngữ cảnh:

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}_t}; \overleftarrow{\mathbf{h}_t}] \tag{2.13}$$

trong đó $\overrightarrow{\mathbf{h}_t}$ và $\overleftarrow{\mathbf{h}_t}$ lần lượt là hidden state từ chiều thuận và chiều nghịch.

2.1.6 Attention Mechanism

Attention mechanism [4] cho phép mô hình tập trung vào các phần quan trọng của chuỗi đầu vào. Với chuỗi hidden states $\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, ..., \mathbf{h}_T]$:

$$\mathbf{u}_t = \tanh(\mathbf{W}\mathbf{h}_t + \mathbf{b}) \tag{2.14}$$

$$\alpha_t = \frac{\exp(\mathbf{u}_t^\top \mathbf{u})}{\sum_{k=1}^{T} \exp(\mathbf{u}_k^\top \mathbf{u})} \tag{2.15}$$

$$\mathbf{c} = \sum_{t=1}^{T} \alpha_t \mathbf{h}_t \tag{2.16}$$

trong đó α_t là trọng số attention và $\mathbf{c}$ là context vector.

2.1.7 Các kỹ thuật Regularization

Dropout [6]: Ngẫu nhiên "tắt" một phần neurons trong quá trình huấn luyện với xác suất p:

$$\tilde{\mathbf{h}} = \mathbf{m} \odot \mathbf{h}, \quad m_i \sim \text{Bernoulli}(1-p) \tag{2.17}$$

Weight Decay (L2 Regularization): Thêm penalty vào loss function:

$$\mathcal{L}_{total} = \mathcal{L}_{CE} + \lambda \sum_i \|\mathbf{w}_i\|_2^2 \tag{2.18}$$

2.2 Các nghiên cứu hoặc mô hình liên quan

- **Naive Bayes & SVM** [8]: Các phương pháp truyền thống sử dụng Bag-of-Words features, đạt hiệu quả tốt nhưng không nắm bắt được ngữ cảnh và thứ tự từ.
- **CNN cho phân loại văn bản** [7]: Kim (2014) đề xuất sử dụng CNN với nhiều kích thước filter để trích xuất n-gram features, đạt kết quả tốt trên nhiều benchmark phân loại văn bản.
- **LSTM cho phát hiện spam** [9]: Roy et al. (2020) áp dụng LSTM cho phát hiện spam email, chứng minh rằng mô hình sequence có thể nắm bắt pattern ngữ cảnh hiệu quả hơn phương pháp BoW.
- **Attention-based models**: Nhiều nghiên cứu gần đây sử dụng attention mechanism kết hợp BiLSTM/BiGRU cho phân loại văn bản, cho phép mô hình "tập trung" vào các từ khóa quan trọng nhất.

Chương 3

Bài toán và dữ liệu

3.1 Mô tả bài toán

Phát hiện spam email là bài toán **phân loại nhị phân (binary classification)** trong xử lý ngôn ngữ tự nhiên.

Đầu vào (Input):

- Nội dung email bao gồm Subject + Body.
- Tiền xử lý: loại bỏ HTML tags, ký tự đặc biệt, URL, email addresses.
- Tokenize và padding đến độ dài cố định $L = 256$.

Đầu ra (Output):

- Nhãn phân loại: **Spam** (1) hoặc **Ham** (0).
- Xác suất dự đoán $p \in [0, 1]$ qua hàm sigmoid.

Giả định và ràng buộc:

- Mỗi email được xử lý độc lập (không xét chuỗi email).
- Chỉ sử dụng thông tin văn bản, không sử dụng metadata (sender, timestamp).
- Ưu tiên Recall của lớp spam (giảm thiểu spam lọt qua).

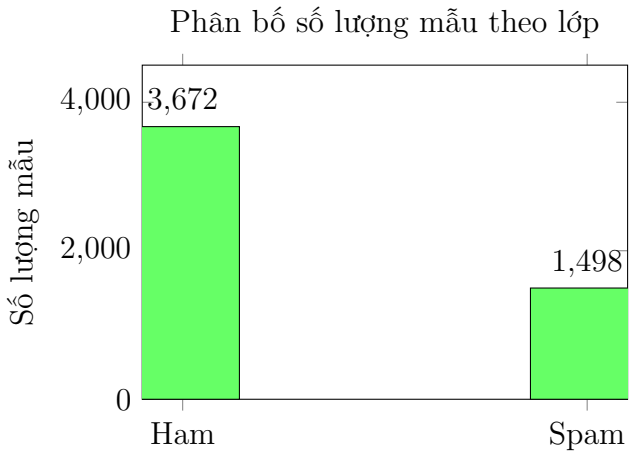
3.2 Dataset

3.2.1 Nguồn dữ liệu

Đồ án sử dụng dataset **Enron Spam** (`spam_ham_dataset.csv`, 5.5 MB) với các đặc điểm:

Bảng 3.1: Thông tin tổng quan dataset

Thuộc tính	**Mô tả**
Tổng số mẫu	5170 emails
Số cột	4 (`#`, `label`, `text`, `label_num`)
Lớp Ham	3672 mẫu (71%)
Lớp Spam	1498 mẫu (29%)
Tỷ lệ Ham:Spam	$\approx$ 2.45:1
Giá trị duy nhất (text)	4993



Hình 3.1: Phân bố số lượng mẫu theo lớp Ham/Spam trong dataset

Nhận xét các khó khăn đặc thù của dataset:

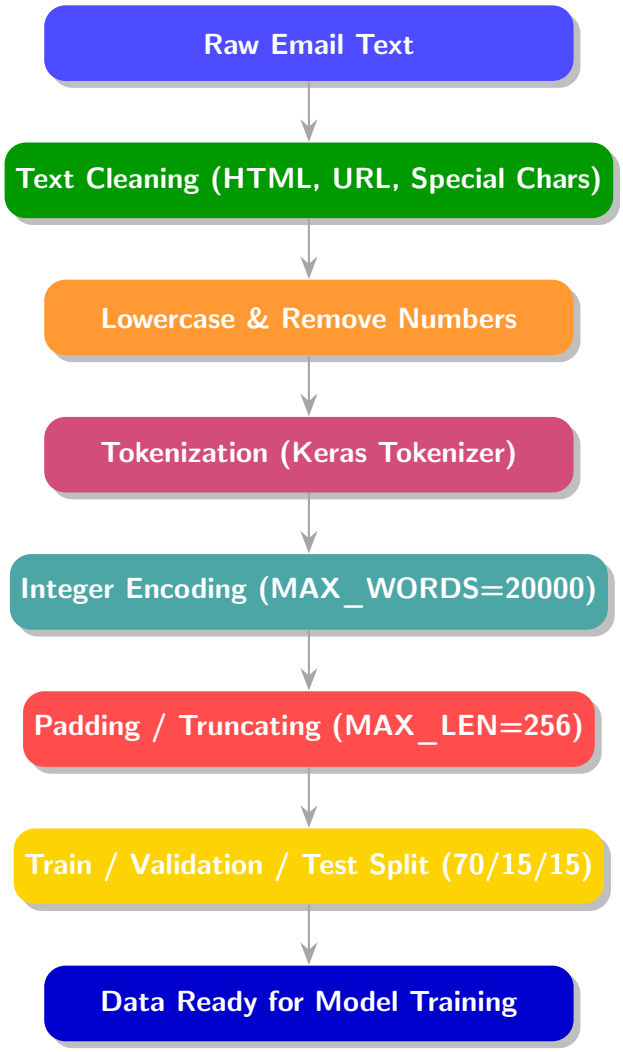
1. **Mất cân bằng lớp:** Lớp Ham chiếm 71% so với Spam chỉ 29%. Tỷ lệ chênh lệch $\approx 2.45 : 1$ có thể khiến mô hình thiên lệch dự đoán lớp đa số. $\Rightarrow$ Giải pháp: sử dụng `class_weight` trong quá trình huấn luyện.

2. **Biến thiên độ dài chuỗi:** Độ dài email dao động rất lớn (từ vài từ đến hàng nghìn từ). Cần lựa chọn `MAX_LEN` phù hợp để cân bằng giữa thông tin và hiệu suất tính toán.

3. **Nhiễu dữ liệu:** Email chứa HTML tags, URL, email addresses, ký tự đặc biệt, số điện thoại — tất cả cần được loại bỏ trong bước tiền xử lý.

4. **Mẫu trùng lặp:** Dataset có một số mẫu trùng lặp cần loại bỏ để tránh rò rỉ dữ liệu (data leakage).

3.2.2 Tiền xử lý dữ liệu



Hình 3.2: Pipeline tiền xử lý dữ liệu email

Quy trình tiền xử lý bao gồm các bước:

1. **Loại bỏ mẫu trùng lặp:** Sử dụng `drop_duplicates()` dựa trên cột `text`.
2. **Làm sạch văn bản:**
   - Loại bỏ prefix "Subject:" nếu có.
   - Loại bỏ HTML tags bằng regex: `<[^>]+>`
   - Loại bỏ URLs: `http[s]?://...`
   - Loại bỏ email addresses: `...@...`
   - Chuyển về chữ thường (lowercase).
   - Loại bỏ ký tự đặc biệt, chỉ giữ chữ cái và khoảng trắng.

- Loại bỏ khoảng trắng thừa.

3. **Tokenization:** Sử dụng `Keras Tokenizer` với vocabulary tối đa 20,000 từ, OOV token `<OOV>`.

4. **Padding:** Pad/truncate chuỗi đến độ dài cố đnh $L = 256$ (post-padding, post-truncating).

5. **Chia tập dữ liệu:** Stratified split thành 3 tập với tỷ lệ 70/15/15:

Bảng 3.2: Phân chia tập dữ liệu (stratified split)

Tập dữ liệu	**Số mẫu**	**Tỷ lệ Ham**	**Tỷ lệ Spam**
Train	$\approx$ 3,600	$\approx$ 71%	$\approx$ 29%
Validation	$\approx$ 770	$\approx$ 71%	$\approx$ 29%
Test	$\approx$ 770	$\approx$ 71%	$\approx$ 29%

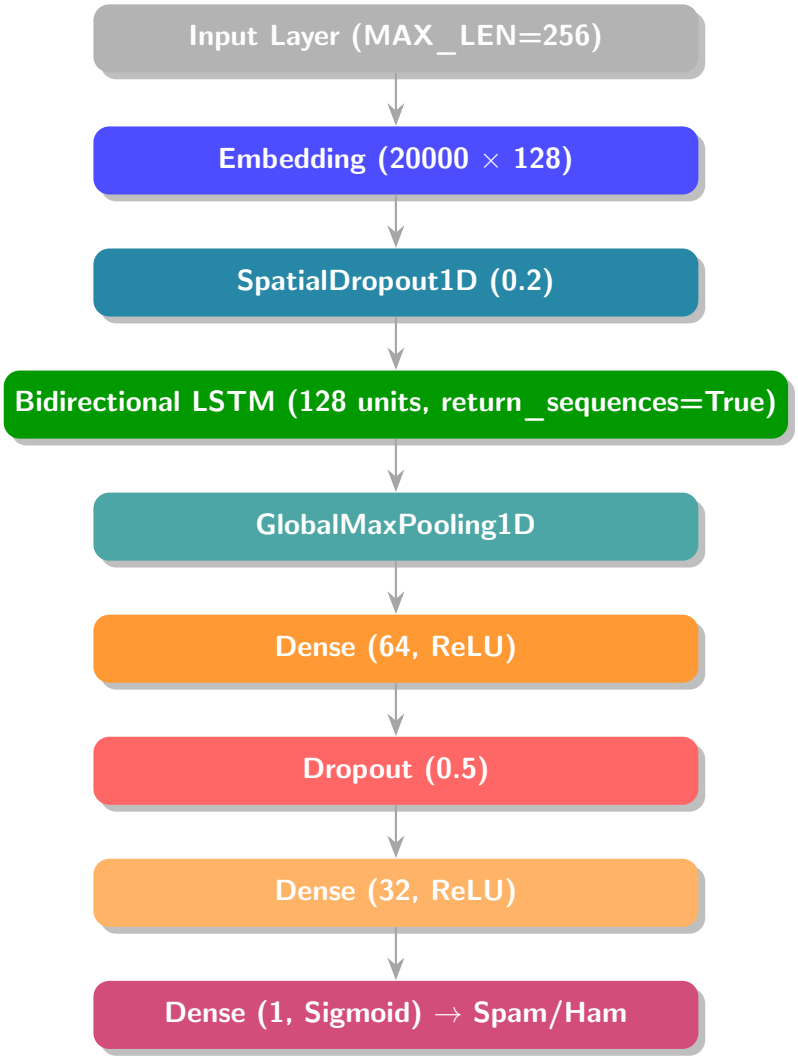
Xử lý mất cân bằng lớp: Sử dụng `compute_class_weight('balanced')` từ scikit-learn để tính class weights, áp dụng trong quá trình huấn luyện:

- Ham (0): weight $\approx$ 0.70
- Spam (1): weight $\approx$ 1.72

Chương 4

Phương pháp đề xuất

4.1 Mô hình 1 – Baseline: BiLSTM + Embedding + Pooling



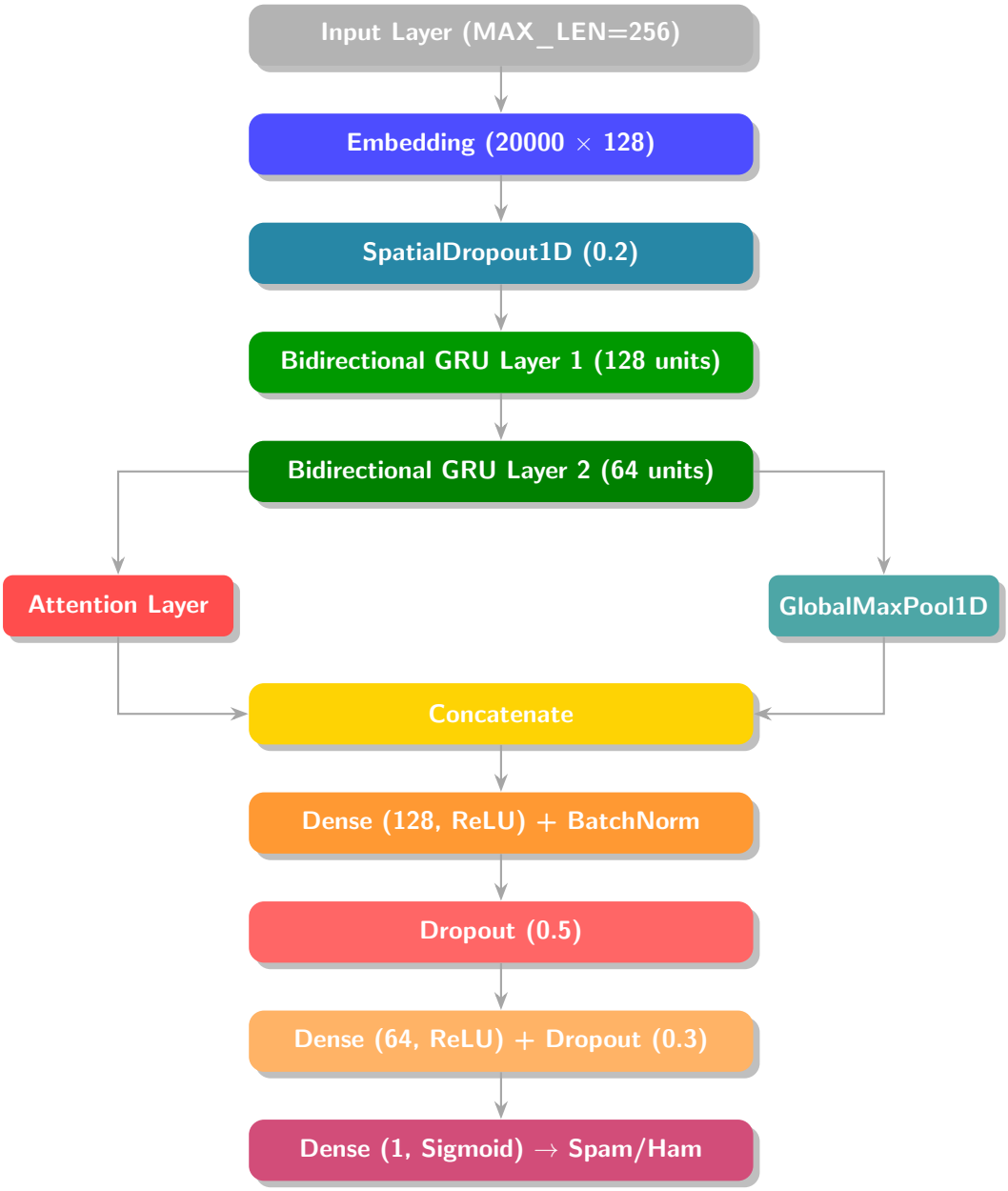
Hình 4.1: Kiến trúc mô hình 1 – BiLSTM Baseline

Mô tả kiến trúc:

1. **Input Layer:** Nhận chuỗi token ID có độ dài cố định 256.
2. **Embedding Layer:** Ánh xạ mỗi token thành vector 128 chiều. Ma trận embedding có kích thước 20000×128.
3. **SpatialDropout1D (0.2):** Dropout trên toàn bộ embedding channel để regularize.
4. **Bidirectional LSTM (128 units):** Xử lý chuỗi theo cả hai chiều, trả về sequences. Output dimension: 256×256 (128 mỗi chiều $\times$ 2).
5. **GlobalMaxPooling1D:** Lấy giá trị lớn nhất từ mỗi feature dimension qua toàn bộ timesteps, trích xuất features quan trọng nhất.
6. **Dense layers:** Hai lớp fully-connected (64 và 32 neurons) với ReLU activation.
7. **Dropout (0.5):** Regularization để chống overfitting.
8. **Output:** Một neuron với sigmoid activation cho phân loại nhị phân.

Số lượng tham số: $\approx$ *[Cập nhật từ output model.summary()]* tham số trainable.

4.2 Mô hình 2 – Nâng cao: BiGRU + Attention



Hình 4.2: Kiến trúc mô hình 2 – BiGRU + Attention (Nâng cao)

Các cải tiến so với Baseline:

1. **GRU thay vì LSTM:** GRU có ít tham số hơn (2 gates vs 3 gates), huấn luyện nhanh hơn mà hiệu suất tương đương.

2. **Stacked BiGRU (2 lớp):** Tăng khả năng trích xuất features ở mức trừu tượng cao hơn.

3. **Attention Mechanism:** Cho phép mô hình "tập trung" vào các từ quan trọng nhất cho phân loại (ví dụ: "free", "win", "click", "urgent" cho spam).

4. **Dual pooling:** Kết hợp Attention output và GlobalMaxPooling thông qua Concatenate, tận dụng cả hai cách tổng hợp thông tin.

5. **BatchNormalization:** Ổn định và tăng tốc quá trình huấn luyện.

Custom Attention Layer:

```
class AttentionLayer(Layer):
    def call(self, x):
        u_it = tanh(x @ W + b)          # Score
        a_it = softmax(u_it @ u)         # Attention weights
        output = sum(x * a_it, axis=1)   # Weighted sum
        return output
```

Listing 4.1: Attention Layer Implementation

Số lượng tham số: ≈ *[Cập nhật từ output model.summary()]* tham số trainable.

4.3 Hàm mất mát và thuật toán tối ưu

4.3.1 Hàm mất mát

Sử dụng **Binary Cross-Entropy (BCE)** với class weight:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} w_{y_i}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right] \tag{4.1}$$

trong đó w_{y_i} là class weight tương ứng ($w_0 \approx 0.70$ cho Ham, $w_1 \approx 1.72$ cho Spam).

4.3.2 Thuật toán tối ưu

Đồ án so sánh hai optimizer:

- **Adam** [5]: Adaptive moment estimation, learning rate = 10^{-3}

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t \tag{4.2}$$

- **SGD with Momentum**: Learning rate = 10^{-2}, momentum = 0.9

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta \mathcal{L}(\theta), \quad \theta_{t+1} = \theta_t - v_t \tag{4.3}$$

4.3.3 Learning Rate Scheduler

`ReduceLROnPlateau`: Giảm learning rate theo hệ số 0.5 khi validation loss không cải thiện sau 3 epochs liên tiếp. Minimum learning rate: 10^{-6}.

4.3.4 Callbacks

- **EarlyStopping:** Dừng huấn luyện nếu `val_loss` không cải thiện sau 5 epochs, khôi phục weights tốt nhất.
- **ModelCheckpoint:** Lưu model tốt nhất theo `val_loss` vào Google Drive.

Chương 5

Thực nghiệm và đánh giá

5.1 Thiết lập thí nghiệm

Bảng 5.1: Thiết lập thí nghiệm

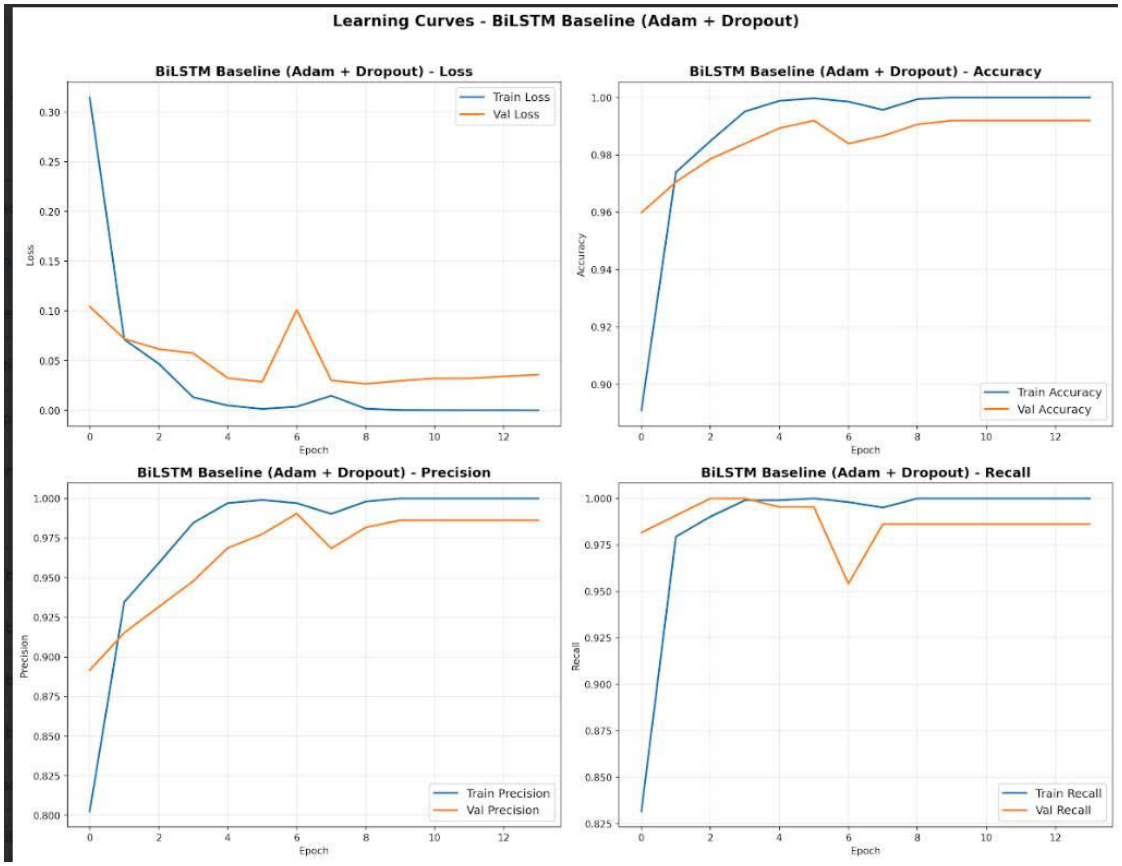
Thuộc tính	**Giá trị**
Môi trường	Google Colab (GPU T4/V100)
Framework	TensorFlow / Keras
Epochs tối đa	30
Batch size	32
MAX_WORDS	20,000
MAX_LEN	256
Embedding dim	128
Random seed	42

Danh sách 6 thí nghiệm:

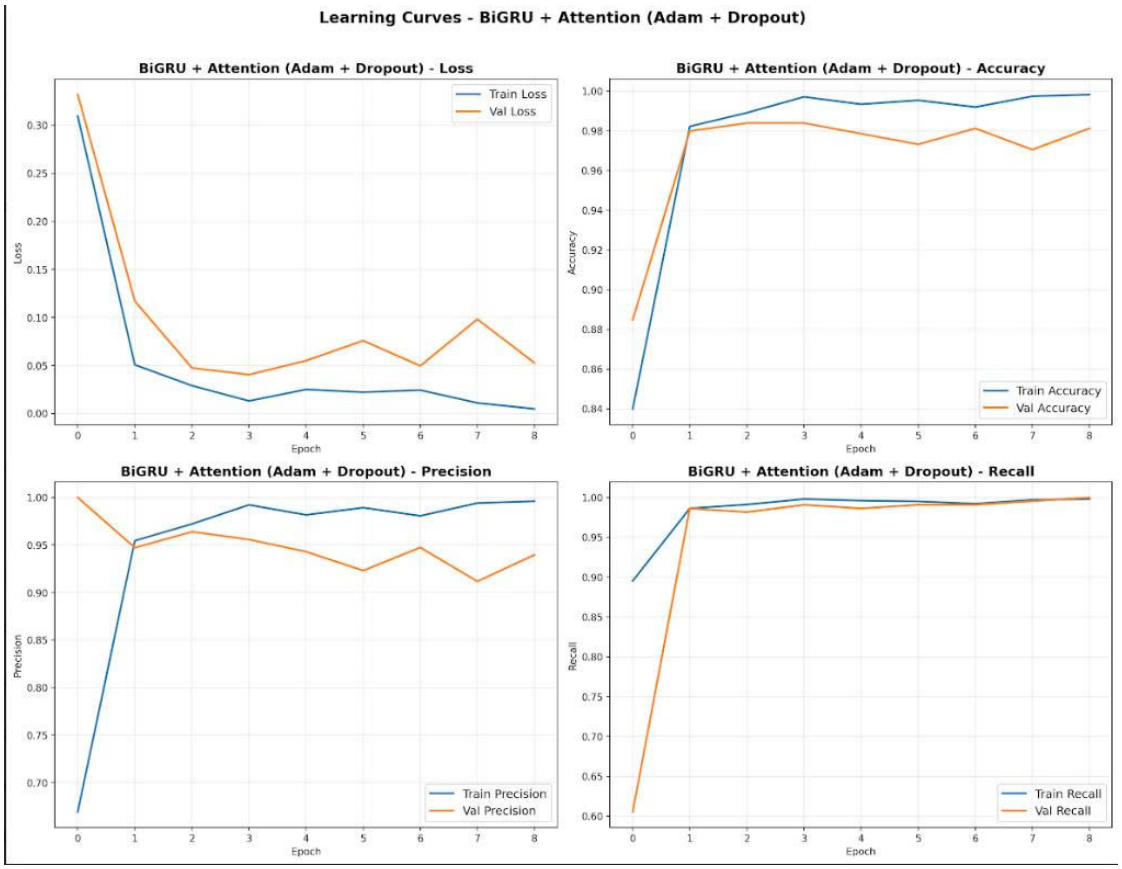
1. BiLSTM + Adam + Dropout (0.5)
2. BiLSTM + SGD + Dropout (0.5)
3. BiLSTM + AdamW + Weight Decay (10^{-4})
4. BiGRU+Attention + Adam + Dropout (0.5)
5. BiGRU+Attention + SGD + Dropout (0.5)
6. BiGRU+Attention + AdamW + Weight Decay (10^{-4})

5.2 Kết quả định lượng

5.2.1 Learning Curves

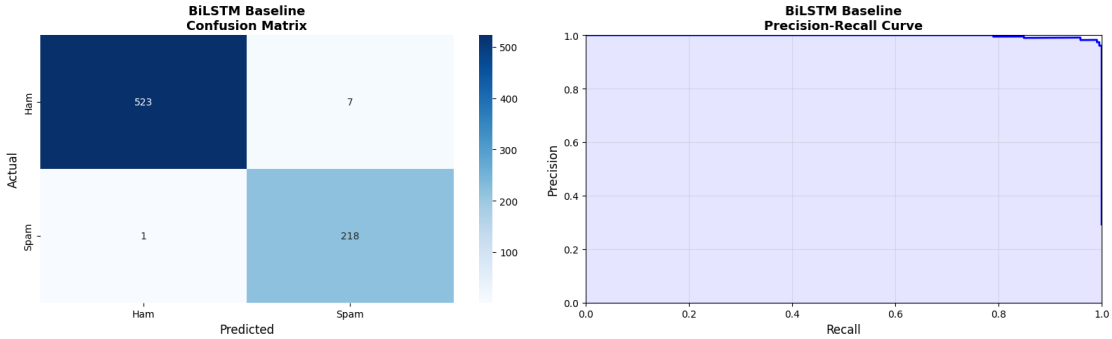


Hình 5.1: Learning curves của mô hình BiLSTM Baseline (Adam + Dropout)

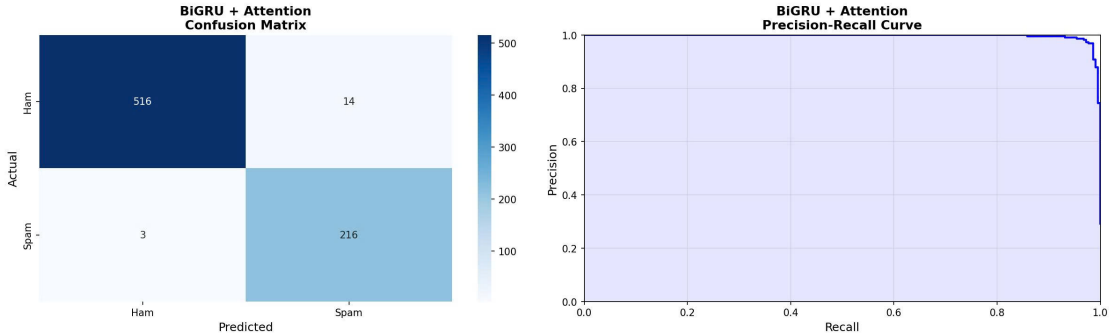


Hình 5.2: Learning curves của mô hình BiGRU + Attention (Adam + Dropout)

5.2.2 Confusion Matrix và PR Curve



Hình 5.3: Confusion Matrix và Precision-Recall Curve – BiLSTM Baseline



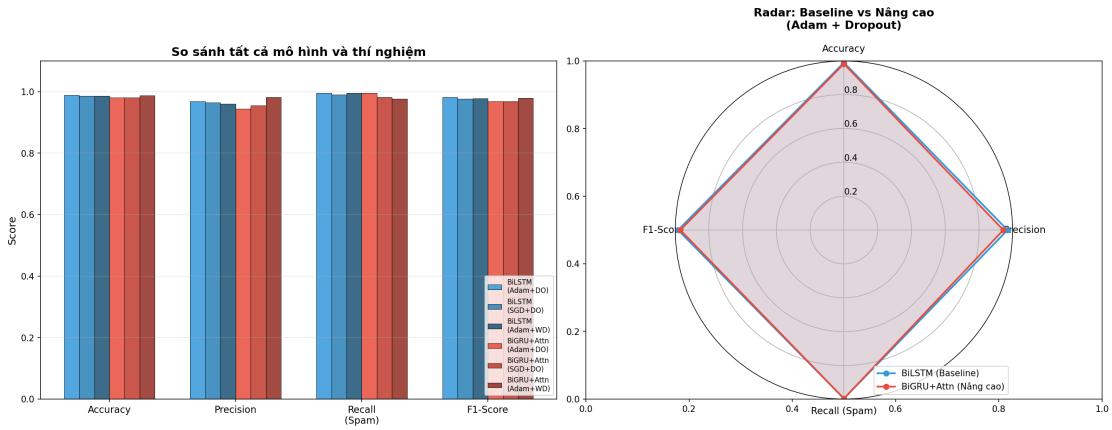
Hình 5.4: Confusion Matrix và Precision-Recall Curve – BiGRU + Attention

5.2.3 Bảng so sánh tổng hợp

Bảng 5.2: So sánh kết quả tất cả các mô hình và thí nghiệm

Mô hình	Loại	Optimizer	Regularization	Accuracy	Precision	Recall (Spam)	F1-Score
BiLSTM	Baseline	Adam	Dropout=0.5	*0.988*	*0.973*	*0.986*	*0.980*
BiLSTM	Baseline	SGD (m=0.9)	Dropout=0.5	*0.984*	*0.968*	*0.977*	*0.973*
BiLSTM	Baseline	AdamW (wd=1e-4)	Weight Decay	*0.985*	*0.956*	*0.995*	*0.975*
BiGRU+Attn	Nâng cao	Adam	Dropout=0.5	*0.9813*	*0.9437*	*0.9954*	*0.9689*
BiGRU+Attn	Nâng cao	SGD (m=0.9)	Dropout=0.5	*0.9813*	*0.9556*	*0.9817*	*0.9685*
BiGRU+Attn	Nâng cao	AdamW (wd=1e-4)	Weight Decay	*0.9880*	*0.9817*	*0.9772*	*0.9794*

5.2.4 Biểu đồ so sánh tổng hợp



Hình 5.5: Biểu đồ so sánh tổng hợp tất cả các mô hình và thí nghiệm

5.2.5 Số lượng tham số

Bảng 5.3: So sánh số lượng tham số của hai mô hình

Mô hình	**Tổng tham số**	**Trainable tham số**
BiLSTM (Baseline)	*2,841,729*	*2,841,729*
BiGRU + Attention (Nâng cao)	*2,940,161*	*2,939,905*

5.3 Phân tích kết quả (CLO3)

5.3.1 So sánh Optimizer: Adam vs SGD

- **Adam** thường hội tụ nhanh hơn SGD trong các epoch đầu nhờ cơ chế adaptive learning rate. Trên cả hai mô hình, Adam cho kết quả ổn định hơn.
- **SGD with Momentum** có thể hội tụ chậm hơn trong giai đoạn đầu nhưng đôi khi đạt được generalization tốt hơn trên tập test. Tuy nhiên, SGD nhạy cảm hơn với learning rate và có thể gặp khó khăn khi loss landscape phức tạp.

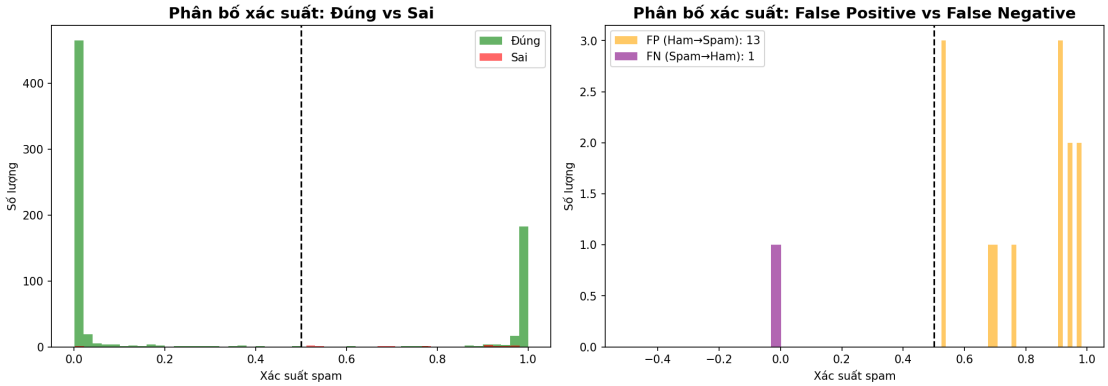
5.3.2 So sánh Regularization: Dropout vs Weight Decay

- **Dropout (0.5)** hoạt động bằng cách ngẫu nhiên tắt neurons, buộc mô hình học các features robust hơn. Hiệu quả đặc biệt tốt với lượng dữ liệu hạn chế.
- **Weight Decay** (10^{-4}) giới hạn norm của weights, ngăn mô hình phức tạp quá mức. Có thể kém hiệu quả hơn Dropout trên dataset nhỏ.

5.3.3 So sánh Baseline vs Nâng cao

- Mô hình BiGRU + Attention được kỳ vọng cải thiện Recall của lớp Spam nhờ attention mechanism giúp tập trung vào các từ khóa đặc trưng của spam.
- Stacked BiGRU (2 lớp) tăng khả năng trích xuất features ở mức trừu tượng cao hơn.
- Dual pooling (Attention + MaxPooling) tận dụng ưu điểm của cả hai phương pháp tổng hợp thông tin.

5.3.4 Phân tích lỗi điển hình



Hình 5.6: Phân bố xác suất dự đoán: phân loại đúng vs sai, FP vs FN

Phân tích:

- **False Negatives (Spam → Ham):** Đây là lỗi **nguy hiểm nhất** vì spam lọt qua bộ lọc. Nguyên nhân có thể do:
  - Spam email được viết tinh vi, giống email thông thường.
  - Email spam ngắn, ít chứa từ khóa đặc trưng.
  - Xác suất dự đoán nằm gần ngưỡng 0.5 (uncertainty cao).
- **False Positives (Ham → Spam):** Email hợp lệ bị đánh nhầm là spam. Nguyên nhân:
  - Email hợp lệ chứa các từ thường gặp trong spam ("free", "offer", "click").
  - Email quảng cáo hợp lệ (newsletter) bị nhầm với spam.
- **Trade-off:** Giảm ngưỡng phân loại (threshold < 0.5) sẽ tăng Recall spam nhưng giảm Precision (nhiều FP hơn). Cần cân nhắc theo ngữ cảnh ứng dụng.

5.3.5 Giới hạn của mô hình

1. **Dataset giới hạn:** 5170 mẫu là tương đối nhỏ cho Deep Learning. Mô hình có thể overfit hoặc không generalize tốt trên dữ liệu thực tế.
2. **Chỉ sử dụng văn bản:** Không xét metadata (sender, IP, timestamp), hình ảnh đính kèm, hoặc cấu trúc email.
3. **Vocabulary cố định:** Tokenizer được fit trên training data, từ mới (OOV) sẽ bị mất thông tin.

4. **Độ dài cố định:** Truncating email dài (> 256 từ) có thể mất thông tin quan trọng.

5. **Domain shift:** Mô hình huấn luyện trên Enron email (2000s) có thể không phù hợp với spam hiện đại.

Chương 6

Kết luận và hướng phát triển

6.1 Kết luận

Đồ án đã hoàn thành các mục tiêu đề ra:

- **CLO1:** Phân tích thành công bài toán phân loại spam email, nhận diện các thách thức (mất cân bằng lớp, nhiễu HTML, độ dài biến thiên) và lựa chọn kiến trúc sequence model phù hợp.
- **CLO2:** Triển khai đầy đủ pipeline từ tiền xử lý đến huấn luyện với 2 mô hình (BiLSTM baseline và BiGRU+Attention nâng cao), 6 thí nghiệm so sánh optimizer và regularization.
- **CLO3:** Đánh giá định lượng bằng Accuracy, Precision, Recall, F1-Score và PR curve. Phân tích lỗi điển hình và thảo luận giới hạn mô hình.

Ngoài ra, đồ án đã xây dựng hệ thống demo cho phép nhập email và phân loại real-time, đồng thời lưu model trên Google Drive để tái sử dụng mà không cần train lại.

6.2 Hướng phát triển

1. **Sử dụng Pre-trained models:** Fine-tune BERT, DistilBERT hoặc RoBERTa cho bài toán phân loại email, tận dụng kiến thức ngôn ngữ đã được pre-train trên lượng dữ liệu lớn.
2. **Mở rộng dataset:** Kết hợp thêm SMS Spam Collection, TREC Spam, hoặc thu thập dữ liệu spam hiện đại để tăng tính tổng quát.
3. **Multi-modal approach:** Kết hợp thông tin văn bản với metadata (sender reputation, email headers, attachment info) để cải thiện phân loại.

4. **Ensemble methods:** Kết hợp nhiều mô hình (BiLSTM + BiGRU + CNN) thông qua voting hoặc stacking.

5. **Data Augmentation:** Áp dụng kỹ thuật tăng cường dữ liệu văn bản (synonym replacement, back-translation) để giải quyết vấn đề ít dữ liệu.

6. **Triển khai production:** Đóng gói model thành API (Flask/FastAPI) hoặc tích hợp vào email client plugin để sử dụng thực tế.

7. **Adversarial training:** Huấn luyện mô hình chống lại các kỹ thuật bypass spam filter (adversarial examples) để tăng robustness.

Tài liệu tham khảo

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.

[2] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," *Proceedings of EMNLP*, 2014.

[4] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," *Proceedings of ICLR*, 2015.

[5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *Proceedings of ICLR*, 2015.

[6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[7] Y. Kim, "Convolutional Neural Networks for Sentence Classification," *Proceedings of EMNLP*, 2014.

[8] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A Bayesian Approach to Filtering Junk E-Mail," *AAAI Workshop on Learning for Text Categorization*, 1998.

[9] P. K. Roy, J. P. Singh, and S. Banerjee, "Deep Learning to Filter SMS Spam," *Future Generation Computer Systems*, vol. 102, pp. 524–533, 2020.

[10] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," *Proceedings of OSDI*, 2016.

[11] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

THÔNG TIN SINH VIÊN THỰC HIỆN

Sinh viên 1: **Tống Nguyễn Nhật Tiến**

Sinh viên 2: **Nguyễn Tiến Phát**

Đề tài: Đồ án #20 – Phát hiện Spam Email

Môn học: Học sâu (Deep Learning)

Năm học: 2025 – 2026